Image Classification with Convolutional Neural Networks on CINIC-10

Pawel Pozorski[1], *Pawel Florek*[1]

[1] Warsaw University of Technology, Poland

{pawel.pozorski, pawel.florek}.stud@pw.edu.pl

Abstract

This paper presents a comparative study of convolutional neural network pipelines for image classification on CINIC-10. We evaluate lightweight and high-capacity backbones, including MobileNetV3 Small, SqueezeNet, ResNet-18, DenseNet-121, and a differentiable NAS-CNN variant. We also test ConvKAN layers as convolutional replacements in efficient models. The study includes a controlled hyperparameter search, modern augmentation methods, reduced-data training, and few-shot metric learning with Prototypical Networks. Experiments are designed for reproducibility with fixed seeds and unified training scripts. We report benchmark context, evaluation protocol, and implementation details that support transparent comparison across model families and training regimes.

Index Terms: image classification, convolutional neural networks, CINIC-10, neural architecture search, few-shot learning

1. Introduction

Convolutional neural networks (CNNs) [1] remain a practical foundation for image recognition [2], but model selection is highly task- and constraint-dependent [3, 4]. In this project, we study how different CNN paradigms perform on CINIC-10 [5], a dataset that preserves CIFAR-like [6] image size while increasing distributional complexity via ImageNet-derived [7] samples.

Compared with CIFAR-10, CINIC-10 introduces a stronger distribution shift because class semantics are preserved while image statistics are broadened by downsampled ImageNet content [5]. This makes the benchmark less prone to narrow in-distribution overfitting and more informative for robustness-oriented evaluation. In practice, models that perform similarly on CIFAR-like data may diverge on CINIC-10 once they are exposed to larger intra-class variation, background diversity, and texture/illumination changes [5, 8].

Our goals are threefold: (i) compare efficient and high-capacity architectures under a common training framework, (ii) evaluate whether ConvKAN-style convolutions improve compact models, and (iii) analyze robustness in low-data conditions using subset training and few-shot learning.

2. Methodology

2.1. Dataset

CINIC-10 [5] contains 270,000 RGB images at 32×32 resolution and 10 balanced classes [5]. Data is organized into train, validation, and test splits of 90,000 samples each. The inclusion of ImageNet-derived examples increases intra-class variation relative to CIFAR-10 and makes generalization analysis more informative [5].

2.2. Model families

We evaluate three groups:

- **Search-based architecture**: a differentiable one-shot NAS-CNN supernet with architecture parameters optimized jointly with weights [9].
- **Efficient backbones**: MobileNetV3 Small [10] and SqueezeNet [11], including ConvKAN substitutions [12].
- **Transfer baselines**: ResNet-18 [13] and DenseNet-121 [14] adapted for the CINIC-10 label space.

For ConvKAN variants, we replace eligible spatial convolution layers with `torchkan`-provided [15] ConvKAN layers while preserving layer shape and stride constraints. Conceptually, this swaps standard linear convolutional kernels for KAN-style parameterization with learnable nonlinear basis behavior on connections, motivated by recent work on Kolmogorov-Arnold Networks [16, 12]. Unlike standard convolutions, which apply fixed scalar weights followed by a static, node-wise activation function (e.g., $y = \tanh(\sum w_i x_i)$), KANs shift the nonlinearity directly to the network edges. In a ConvKAN layer, the scalar elements of a traditional convolutional kernel are replaced by learnable 1D nonlinear functions, typically parameterized as B-splines. Consequently, each connection applies a dynamic, finely-tuned activation $\phi(x)$ to the input pixels within the receptive field, and the output node simply sums these transformed signals ($y = \sum \phi_i(x_i)$). This theoretical increase in per-parameter expressivity aims to allow lightweight architectures to capture complex spatial patterns more efficiently than standard linear filters.

For NAS, our supernet is constructed as a sequence of searchable edges, formulated via a continuous relaxation of the discrete architecture space [17]. Instead of selecting a single operation, the output of each edge during the search phase is computed as a weighted sum of all candidate operations $o \in \mathcal{O}$, combined using a softmax distribution over learnable architecture parameters α:

$$\bar{o}(x) = \sum_{o \in \mathcal{O}} \frac{\exp(\alpha_o)}{\sum_{o'} \exp(\alpha_{o'})} o(x)$$

Our specific search space $\mathcal{O}$ consists of 5 candidate operations designed to balance capacity and efficiency: standard convolutions (`conv3x3`, `conv5x5`), parameter-efficient `depthwise3x3`[1], spatial reduction `maxpool3x3_proj`,

[1] A depthwise convolution does not share weights across channels - it learns a distinct spatial filter for each input channel. For example, given an input with 32 channels, a standard 3×3 convolution computing 32 output channels requires $32 \times 32 \times 3 \times 3 = 9{,}216$ parameters to mix all spatial and channel information simultaneously. A depthwise 3×3 layer requires only $32 \times 3 \times 3 = 288$ parameters, as it applies exactly one unique 3×3 filter to each input channel independently.

and identity mapping `skip_proj`. The `_proj` variants explicitly incorporate a 1×1 pointwise convolution to ensure tensor dimensions match when spatial strides or channel counts shift across an edge.

The macro-architecture skeleton restricts the search to 6 sequential edges, following a standard hierarchical feature pyramid defined by these input/output channel ($C_{in} \rightarrow C_{out}$) and stride ($s$) transitions:

- $(3 \rightarrow 32, s = 1)$, $(32 \rightarrow 32, s = 1)$, $(32 \rightarrow 64, s = 2)$,
- $(64 \rightarrow 64, s = 1)$, $(64 \rightarrow 128, s = 2)$, $(128 \rightarrow 128, s = 1)$.

This formulation enables the joint gradient-based optimization of both the network weights and the architecture parameters α. Following the search stage, the continuous edge mixtures are discretized by taking the $\arg\max$ of the architecture logits to derive the final feature transformation and downsampling pattern.

2.3. Training and search setup

A structured grid search is used for MobileNetV3 Small with the Cartesian product of:

- optimizer $\in$ {SGD [18], AdamW [19]},
- batch size $\in \{128, 256\}$,
- dropout $\in \{0.0, 0.1, 0.5\}$,
- weight decay $\in \{0.0, 10^{-4}\}$.

These bounds were selected to bracket key training regimes: batch sizes of 128 and 256 balance computational efficiency with the implicit regularization of stochastic mini-batch noise [20], while the dropout values evaluate whether a lightweight architecture requires no explicit regularization (0.0), mild feature decorrelation (0.1), or classic aggressive dropout (0.5) [21]. This yields 24 configurations per seed in Stage 1 (refer to Section 4.1 for details). This stage is repeated for three fixed seeds $\{0, 42, 3407\}$, resulting in 72 total runs; thus, seed is treated as an experimental repeatability axis (for stability statistics), not as a tuned hyperparameter. For NAS optimization, temperature annealing and entropy regularization are applied to architecture logits to move from exploratory mixtures to sharper operation selection.

2.4. Reproducibility and seed control

The same seed policy is applied consistently across all stages in the execution plan (grid search, augmentation ablation, architecture comparison, NAS two-stage, reduced-data training, and few-shot episodes). One top-level seed initializes Python, NumPy [22], and PyTorch [23] RNG states at process start. Dataloader randomness is controlled with split-specific seeded generators (train/validation/test offsets), so ordering is deterministic within a run but decorrelated across splits. In multi-worker loading, each worker receives a deterministic derived seed and forwards it to Python and NumPy RNGs inside that worker process.

This seed is also propagated to class-stratified subset sampling and few-shot episode construction, so data selection, task generation, augmentation sampling, and batch shuffling are all reproducible for a given run directory (e.g., `seed_0`, `seed_42`, `seed_3407`). Reporting across these seeds provides mean/variance-style stability evidence rather than a single-point estimate.

2.5. Data augmentation

The training pipeline includes geometric and photometric transforms and mixing-based regularization:

- random rotation and color jitter,
- MixUp [24]: two samples (x_i, y_i) and (x_j, y_j) are linearly combined as $\tilde{x} = \lambda x_i + (1-\lambda)x_j$ and $\tilde{y} = \lambda y_i + (1-\lambda)y_j$, where the interpolation weight $\lambda \in [0, 1]$ is sampled from a Beta distribution, $\lambda \sim \text{Beta}(\alpha, \alpha)$. This continuous mixing smooths decision boundaries and reduces overconfident predictions. For our experiments, we set $\alpha = 0.1$ as it is a pytorch-default;
- CutMix [25]: a rectangular patch from one image replaces the corresponding region in another image. The bounding box is determined by a combination ratio λ (sampled from a Beta distribution identically to MixUp), with its center uniformly sampled across the spatial dimensions and its width and height scaled proportionally to $\sqrt{1-\lambda}$. Labels are mixed by the exact visible area ratio, encouraging localized evidence use and improving robustness to occlusion;
- AutoAugment (CIFAR-10 policy) [26]: an augmentation schedule originally discovered via reinforcement learning. For each training sample, one sub-policy is uniformly sampled from a pool of 25 learned policies. Each sub-policy sequentially applies two operations (ranging from geometric transformations like shearing to photometric ones like solarization or color inversion), each with a fixed, optimized probability and magnitude. This automated diversification prevents overfitting more effectively than static, hand-crafted heuristics;

2.6. Low-data evaluation

Two complementary low-resource protocols are used:

- **Subset training**: supervised training on 5% of data per class, sampled with class-stratified selection. For each class, indices are sampled independently without replacement using the experiment seed, with at least one sample retained per class; this preserves class coverage and prevents label-frequency drift that appears in naive global subsampling.
- **Few-shot learning**: episodic training with Prototypical Networks [27]. To optimize the model for transferable class separation rather than closed-set classification, training proceeds through N-way, K-shot episodes. Each episode is constructed and evaluated through the following steps:

  1. **Task sampling**: A support set S is formed by randomly sampling N classes from the training split and K images per class. A disjoint query set Q is then sampled from those exact same N classes to serve as the episodic test set.
  2. **Latent embedding and prototype calculation**: The CNN backbone, parameterized by θ, acts as an embedding function f_θ that maps images into a continuous latent space. For each class c, a prototype vector $\mathbf{v}_c$ is computed as the mean of its K embedded support examples:

$$\mathbf{v}_c = \frac{1}{K} \sum_{(x_i, y_i) \in S_c} f_\theta(x_i)$$

  3. **Distance formulation**: Query images $x_q \in Q$ are passed through the same backbone f_θ. The network evaluates the squared Euclidean distance d between the query embedding $f_\theta(x_q)$ and each class prototype $\mathbf{v}_c$.

4. **Prediction and optimization**: A softmax function over the negative Euclidean distances yields a probability distribution over the N episodic classes:

$$p(y = c|x_q) = \frac{\exp(-d(f_\theta(x_q), \mathbf{v}_c))}{\sum_{c'} \exp(-d(f_\theta(x_q), \mathbf{v}_{c'}))}$$

The model weights θ are updated via backpropagation by minimizing the negative log-probability (cross-entropy loss) of the true query labels.

By continually optimizing this distance-based metric across varying episodic tasks, the network learns a generalized feature space where intra-class samples cluster tightly, making it highly effective for low-data adaptation on unseen data distributions.

3. Related Work and Benchmark Context

3.1. Efficient CNN architectures

Lightweight vision models evolved from compact macro-architectures (e.g., SqueezeNet) toward systematic use of depthwise-separable and inverted bottleneck designs in MobileNet families [11, 28, 10]. Subsequent compound scaling strategies, such as EfficientNet, further formalized the accuracy-efficiency trade-off across depth, width, and resolution [4]. Our efficient-model branch is positioned in this line of work and tests whether ConvKAN substitutions can improve compact backbones under fixed training budgets.

3.2. Differentiable NAS

Differentiable NAS methods treat architecture choice as a continuous relaxation optimized by gradient descent, as popularized by DARTS [17]. Hardware-aware variants such as ProxylessNAS further emphasize practical deployment constraints [29]. Our NAS-CNN setup follows this paradigm by optimizing operation-mixture logits and then retraining a discretized architecture selected from the learned supernet.

3.3. Few-shot learning context

Few-shot learning approaches can be broadly grouped into metric-learning and optimization-based strategies. Prototypical Networks represent the metric-learning family by classifying queries via distances to class prototypes in embedding space [27], whereas MAML and related methods optimize rapid adaptation through meta-learned initialization [30]. We use ProtoNet because it integrates cleanly with the planned episodic pipeline and provides a stable low-data baseline for comparison with reduced-data supervised training.

3.4. CINIC-10 benchmark context

Published CINIC-10 baselines report representative validation error rates around 12.42% for ResNet-18, 8.74% for DenseNet-121 (train+validation protocol), and higher error for lightweight MobileNet variants [5]. These numbers provide an external reference scale for interpreting our results.

4. Experimental Design and Reporting

4.1. Experiment execution plan

Our experiments follow a fixed staged protocol consistent with the project execution plan.

Stage 1 **(grid search on MobileNetV3-Small).** For each seed, we run a 24-configuration grid over optimizer, batch size, dropout, and weight decay. Across three seeds this yields 72 runs in total. The resulting `grid_results.csv` files are used to select `BEST_*` training settings.

Stage 2 **(augmentation ablation).** Using Stage-1 `BEST_*` settings, we compare 4 augmentation configurations on MobileNetV3-Small: (i) geometric+photometric only, (ii) +MixUp, (iii) +CutMix, and (iv) +AutoAugment. The selected policy is exported as `BEST_AUG_EXTRA_ARGS`.

Stage 3 **(architecture comparison).** With fixed `BEST_*` and best augmentation, we train SqueezeNet, ResNet-18, DenseNet-121, and ConvKAN variants, and run NAS two-stage search+retrain per seed. This stage provides the final architecture ranking under a common protocol.

Stage 4 **(final low-data evaluation).** We run 2 final protocols with the chosen final configuration: class-stratified reduced-data supervised training (5% per class) and few-shot episodic training (ProtoNet, 5-way/5-shot/15-query).

4.2. Evaluation criteria

Primary metrics are classification accuracy and error rate on validation and test splits. For model comparison, we additionally track parameter counts, training stability across seeds, and resource usage (CPU, GPU, memory, wall-clock time). For NAS, we report both the final architecture's performance and the search phase's efficiency metrics.

4.3. Reproducibility and compute reporting

All experiments are executed through a unified command interface with explicit seed control. We log per-epoch (or per-episode) wall-clock and CPU time and memory usage to support reproducibility and practical comparison of computational cost.

Experiments are run on macOS with Apple Metal backend enabled through PyTorch MPS execution (*Apple M4 16gb RAM MacBook Pro*). The software stack follows project-pinned requirements: Python ≥ 3.13, PyTorch $\geq 2.8.0$ [23], and torchvision $\geq 0.23.0$ [31]. This context allows interpreting runtime and memory numbers as platform-specific measurements rather than framework-agnostic absolutes. For more specifics, refer to the repository [2].

4.4. Limitations

The complete hyperparameter grid is applied only to MobileNetV3 Small due to computational budget. Results for other families rely on reduced tuning and therefore should be interpreted as strong reference configurations rather than exhaustive optima. Our work is based only on the CINIC-10 dataset, so generalization to other benchmarks or real-world applications may be limited. The NAS search space is relatively small and may not capture all promising architectures. Finally, the few-shot evaluation focuses on Prototypical Networks, so conclusions about low-data performance may not extend to optimization-based meta-learning methods.

5. Conclusion

This study defines a consistent and reproducible framework for comparing CNN architectures on CINIC-10 across efficiency, capacity, and low-data regimes. By combining NAS, ConvKAN

[2] `https://github.com/Pawlo77/cinic10`

variants, transfer baselines, and few-shot methods under shared protocols, the project provides a practical basis for selecting models under different resource and data constraints.

6. References

[1] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[2] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, "A ConvNet for the 2020s," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 11 966–11 976.

[3] S. Bianco, R. Cadene, L. Celona, and P. Napoletano, "Benchmark analysis of representative deep neural network architectures," *IEEE access*, vol. 6, pp. 64 270–64 277, 2018.

[4] M. Tan and Q. V. Le, "Efficientnet: Rethinking model scaling for convolutional neural networks," in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 2019, pp. 6105–6114.

[5] L. N. Darlow, E. J. Crowley, A. Antoniou, and A. J. Storkey, "CINIC-10 is not imagenet or CIFAR-10," *CoRR*, vol. abs/1810.03505, 2018. [Online]. Available: http://arxiv.org/abs/1810.03505

[6] A. Krizhevsky and G. Hinton, "Learning multiple layers of features from tiny images," University of Toronto, Tech. Rep., 2009.

[7] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *2009 IEEE conference on computer vision and pattern recognition*. IEEE, 2009, pp. 248–255.

[8] B. Recht, R. Roelofs, L. Schmidt, and V. Shankar, "Do ImageNet classifiers generalize to ImageNet?" in *International Conference on Machine Learning*. PMLR, 2019, pp. 5389–5400.

[9] S. Salmani Pour Avval, N. D. Eskue, R. M. Groves *et al.*, "Systematic review on neural architecture search," *Artificial Intelligence Review*, vol. 58, no. 1, p. 73, 2025. [Online]. Available: https://doi.org/10.1007/s10462-024-11058-w

[10] A. Howard, M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan, Q. V. Le, and H. Adam, "Searching for mobilenetv3," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, October 2019.

[11] F. N. Iandola, M. W. Moskewicz, K. Ashraf, S. Han, W. J. Dally, and K. Keutzer, "Squeezenet: Alexnet-level accuracy with 50x fewer parameters and <1mb model size," *CoRR*, vol. abs/1602.07360, 2016. [Online]. Available: http://arxiv.org/abs/1602.07360

[12] M. Abd Elaziz, I. Ahmed Fares, and A. O. Aseeri, "Ckan: Convolutional kolmogorov–arnold networks model for intrusion detection in iot environment," *IEEE Access*, vol. 12, pp. 134 837–134 851, 2024.

[13] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2016.

[14] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, July 2017.

[15] S. S. Bhattacharjee, "Torchkan," https://github.com/1ssb/torchkan/, 2024.

[16] Z. Liu, Y. Wang, S. Vaidya, F. Ruehle, J. Halverson, M. Soljačić, T. Y. Hou, and M. Tegmark, "Kan: Kolmogorov-arnold networks," *arXiv preprint arXiv:2404.19756*, 2024. [Online]. Available: https://arxiv.org/abs/2404.19756

[17] H. Liu, K. Simonyan, and Y. Yang, "Darts: Differentiable architecture search," in *International Conference on Learning Representations (ICLR)*, 2019, arXiv:1806.09055.

[18] L. Bottou, "Large-scale machine learning with stochastic gradient descent," in *Proceedings of COMPSTAT'2010*. Springer, 2010, pp. 177–186.

[19] I. Loshchilov and F. Hutter, "Fixing weight decay regularization in adam," *CoRR*, vol. abs/1711.05101, 2017. [Online]. Available: http://arxiv.org/abs/1711.05101

[20] D. Masters and C. Luschi, "Revisiting small batch training for deep neural networks," *arXiv preprint arXiv:1804.07612*, 2018.

[21] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: a simple way to prevent neural networks from overfitting," *The journal of machine learning research*, vol. 15, no. 1, pp. 1929–1958, 2014.

[22] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, and T. E. Oliphant, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, Sep. 2020. [Online]. Available: https://doi.org/10.1038/s41586-020-2649-2

[23] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32*. Curran Associates, Inc., 2019, pp. 8024–8035. [Online]. Available: http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf

[24] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, "mixup: Beyond empirical risk minimization," 2018. [Online]. Available: https://arxiv.org/abs/1710.09412

[25] S. Yun, D. Han, S. J. Oh, S. Chun, J. Choe, and Y. Yoo, "Cutmix: Regularization strategy to train strong classifiers with localizable features," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, October 2019.

[26] E. D. Cubuk, B. Zoph, D. Mane, V. Vasudevan, and Q. V. Le, "Autoaugment: Learning augmentation strategies from data," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2019.

[27] J. Snell, K. Swersky, and R. Zemel, "Prototypical networks for few-shot learning," in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol. 30. Curran Associates, Inc., 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/cb8da6767461f2812ae4290eac7cbc42-Paper.pdf

[28] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "Mobilenetv2: Inverted residuals and linear bottlenecks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520.

[29] H. Cai, L. Zhu, and S. Han, "Proxylessnas: Direct neural architecture search on target task and hardware," in *International Conference on Learning Representations (ICLR)*, 2019, arXiv:1812.00332.

[30] C. Finn, P. Abbeel, and S. Levine, "Model-agnostic meta-learning for fast adaptation of deep networks," in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017, pp. 1126–1135.

[31] D. Falbel, *torchvision: Models, Datasets and Transformations for Images*, 2026, r package version 0.8.0.9000. [Online]. Available: https://github.com/mlverse/torchvision